Remaining Useful Life Prediction of Aircraft Engines

NASA C-MAPSS FD001 · AML Course Project, Phase 1

RUL: cycles left before an engine fails

Why RUL matters

Fix too early → good parts are wasted
Fix too late → risk a breakdown
Know the RUL → fix it just in time

GOAL

- Predict RUL from sensor readings
- Compare linear models with tree models
- Report early vs late error and find the "gap"

1 Data	→	2 Features	→	3 Evaluation	→	4 Trees	→	5 Final
RUL label, drop flat columns		10-cycle mean, std, slope		Engine split, linear baseline		Random Forest, boosting		Test once, early vs late, the gap
Aryan		Akhil		Abhayanth		Srijan		Ashrith

Data and RUL label

- Train: 100 engines run to failure. Test: 100 stop early.
- RUL label: last cycle minus current cycle, no cap
- Sensors stay flat, then drift as failure nears
- 8 constant columns removed → 14 useful sensors

14 useful sensors (from 21)

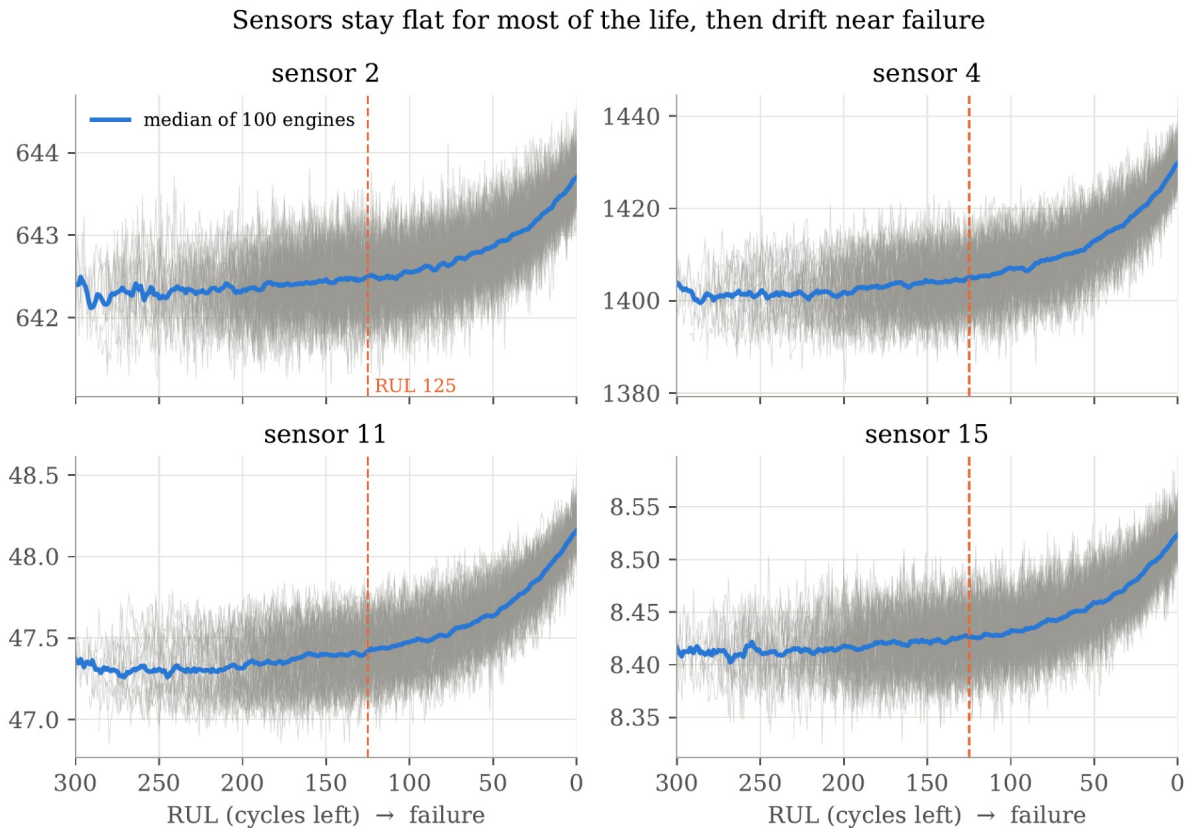


Figure 4 · four sensors vs RUL, 100 training engines

Rolling features

- Each sensor: look at the engine's last 10 cycles
- Mean smooths noise, std shows jumpiness, slope shows wear speed
- Only past cycles, never mixes engines: no leakage
- **Why 10?** Long enough to remove the noise, short enough to work on every engine

57 features: cycle + 14 sensors + 42 rolling

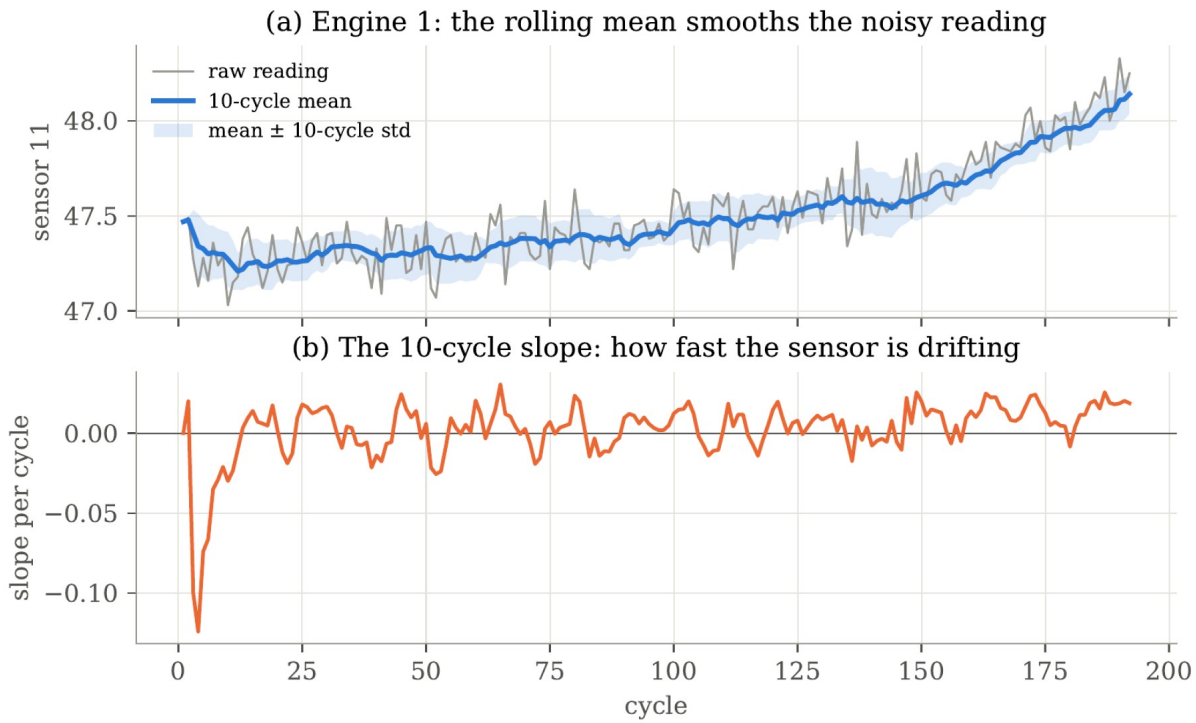


Figure 7 · sensor 11, engine 1: raw vs 10-cycle mean, and slope

Linear baseline: OLS and Elastic Net

- **OLS**: a plain straight-line fit on raw readings, the simplest baseline
- **Elastic Net**: our features + Lasso (drops some) + Ridge (shrinks weights)
- **Why add it**: correlated sensors make OLS unstable; Lasso drops weak features
- **Result**: Elastic Net wins, but both underfit, even on training data

Check	What it tells us	Better
MAE	the average miss, in cycles	Elastic Net
RMSE	like MAE, but big misses count more	Elastic Net
Training error	high for both: a straight line underfits	Neither

Same winner on validation and test · Table 4, Fig. 12

Tree models

- The sensors stay flat for most of the engine's life, then change fast near the end
- Random Forest: many trees averaged → more stable
- Gradient boosting: each tree fixes earlier mistakes → more accurate
- **Near-tie on validation** → pick boosting: safer, overfits less

Check	Random Forest	Gradient boosting
Validation		
MAE	25.0	26.1
RMSE	40.2	40.2
Too optimistic near failure	73%	69%
Error jump, train → val	21 → 43	30 → 43
Test		
MAE	19.0	18.5
RMSE	25.9	25.3

Final evaluation and the gap

- **Best model**: gradient boosting, lowest error and least over-optimistic
- **Late** (near failure): accurate where it matters, but still too optimistic
- **The gap**: far from failure, a 10-cycle window can't see long-term wear
- **The fix**: Phase 2, an LSTM that reads a much longer sensor history

Check	Late	Early
How far off	~18 cycles (trees)	~50 cycles (every model)
Main problem	Too optimistic	Guesses flatten out
Verdict	Good, but risky	The gap → Phase 2

Late = 125 or fewer cycles left · early = more · validation, Table 5

18.5 cycles: average test error (gradient boosting)

Team

Presenter	What they built	Code
Aryan	Data loading, RUL label, EDA	load_data.py, notebook 01
Akhil	Rolling features	features.py, notebook 02
Abhayanth	Engine split, metrics, linear models	evaluation.py, notebook 03
Srijan	Random Forest and gradient boosting	notebook 04
Ashrith	Final evaluation, early/late, the gap	notebook 05

Code: github.com/Abhayanthk/aml-rul-fd001